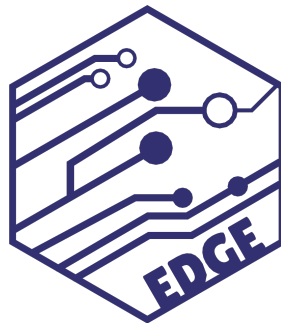




**Politecnico
di Torino**



Lab 0: Profiling & Accelerator Proposal

X-HEEP Accelerator Labs

A custom hardware accelerator design lab

Politecnico di Torino

EDGE Group

Objective

Select one of the four applications below, profile it on X-HEEP, and decide which piece of it you would build custom hardware for. That is the whole lab: no RTL yet, just evidence. The four applications are not toy loops invented for the occasion — each one is a small, integer-only implementation of a kernel that shows up, in a much bigger form, inside a real system described in a real paper. Two of them (`matmul`, `crc32`) are single, homogeneous kernels: profiling tells you “how expensive is this loop”. The other two (`tinyconv`, `attention`) are small *pipelines* of two or three sub-kernels, closer to what a real accelerator design problem looks like: profiling has to tell you *which of several candidate pieces* is worth accelerating, and the honest answer is not always the one that looks the most obviously “hardware-shaped” on paper.

1 The applications

All four live under `sw/applications/<name>/main.c`, already wired with the `PROFILE_START`/`PROFILE_END` macros from `sw/external/profile.h` around the region(s) you should profile. Read the code before you run it — you are going to have to explain, in the proposal, exactly what is happening inside the loop you propose to accelerate.

1.1 `matmul` — integer matrix multiply

A 16×16 integer GEMM (general matrix multiply): the single triple-nested loop below. GEMM is the arithmetic core of essentially every neural-network layer once weights and activations are quantized to low-precision integers for embedded inference [1] — both applications further down this list reduce to GEMM internally. It is also the canonical first accelerator target: regular access pattern, full data reuse, an obvious mapping to a MAC array or systolic array.

```
#define N 16
static int32_t a[N][N], b[N][N], c[N][N];

static void matmul(void) {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            int32_t acc = 0;
            for (int k = 0; k < N; k++)
                acc += a[i][k] * b[k][j];
            c[i][j] = acc;
        }
}
```

1.2 `crc32` — CRC-32 checksum

A bit-serial CRC-32 (the IEEE 802.3 polynomial) over a 256-byte buffer, the classic error-detecting code used in Ethernet frames and countless storage and communication protocols [6]. Unlike `matmul`, this kernel is control-heavy and touches one bit at a time: a poor fit for a wide MAC array, but a very good fit for a small, tightly-coupled instruction extension, or a tiny bit-serial state machine behind a memory-mapped register.

```
static uint32_t crc32(const uint8_t *data, int len) {
    uint32_t crc = 0xFFFFFFFFu;
    for (int i = 0; i < len; i++) {
        crc ^= data[i];
        for (int b = 0; b < 8; b++) {
            uint32_t mask = -(crc & 1u);
            crc = (crc >> 1) ^ (0xEDB88320u & mask);
        }
    }
    return ~crc;
}
```

1.3 tinyconv — a CNN convolution layer

A single Conv2D + ReLU layer: a 3×3 kernel, 2 input channels, 4 output channels, int8 activations/weights with int32 accumulation, over a 10×10 input. This is the same layer structure as the convolutional layers in LeNet-5 [2], the architecture that established the conv+ReLU+pool pattern used by essentially every CNN since, executed here the way an embedded accelerator actually would: as pure integer arithmetic, in the style of integer-only quantized inference [1] rather than floating point.

The kernel is six nested loops instead of three: the outer four walk output channels and output pixels, the inner two slide a 3×3 window over the input. It is still “just” a lot of MACs, but with a *sliding window* access pattern instead of full reuse — the interesting hardware question is how much of that window you can keep resident instead of re-reading it from memory for every output pixel.

```
#define CIN 2
#define COUT 4
#define K 3
#define IH 10
#define IW 10
#define OH (IH - K + 1)
#define OW (IW - K + 1)

static int8_t in[CIN][IH][IW];
static int8_t weight[COUT][CIN][K][K];
static int32_t bias[COUT];
static int32_t out[COUT][OH][OW];

static void conv2d_relu(void) {
    for (int f = 0; f < COUT; f++)
        for (int oy = 0; oy < OH; oy++)
            for (int ox = 0; ox < OW; ox++) {
                int32_t acc = bias[f];
                for (int c = 0; c < CIN; c++)
                    for (int kh = 0; kh < K; kh++)
                        for (int kw = 0; kw < K; kw++)
                            acc += (int32_t)in[c][oy + kh][ox + kw] *
                                (int32_t)weight[f][c][kh][kw];
                out[f][oy][ox] = acc > 0 ? acc : 0; // ReLU
            }
}
```

1.4 attention — single-head scaled dot-product attention

A single attention head, the mechanism at the heart of the Transformer [3], run over a short sequence (8 tokens, model dimension 8) in fixed-point integer arithmetic. Unlike the other three applications, this one is deliberately split into **three** separately profiled sub-kernels, because that is the actual design question a Transformer accelerator faces:

1. `qk_matmul` — scores = $Q \cdot K^\top$ (a GEMM, exactly like `matmul`/).
2. `softmax` — row-wise normalization of the scores into attention weights.
3. `av_matmul` — out = weights $\cdot V$ (a GEMM again).

A real softmax needs an exponential and a division, neither of which is pleasant in integer-only hardware. The implementation below uses a shift-based approximate exponential — decay by a power of two instead of e^x — in the spirit of hardware-oriented softmax designs such as Softmax [4] and integer-only Transformer inference such as I-BERT [5]; it is simplified for teaching and is *not* a faithful reproduction of either paper. The final per-row normalization still uses one integer division per element, which is exactly the kind of detail profiling is for: does that division-heavy loop actually dominate, or do the two matmuls?

```
#define SEQ 8
#define DM 8
```

```
#define WEIGHT_ONE 256 // fixed-point "1.0", Q8
#define DECAY_SHIFT 2

static int8_t Q[SEQ][DM], K[SEQ][DM], V[SEQ][DM];
static int32_t scores[SEQ][SEQ], weights[SEQ][SEQ], out[SEQ][DM];

// 1. scores[i][j] = Q[i] . K[j]
static void qk_matmul(void) {
    for (int i = 0; i < SEQ; i++)
        for (int j = 0; j < SEQ; j++) {
            int32_t acc = 0;
            for (int d = 0; d < DM; d++)
                acc += (int32_t)Q[i][d] * (int32_t)K[j][d];
            scores[i][j] = acc;
        }
}

// 2. row-wise softmax, shift-based integer exponential approximation
static void softmax(void) {
    for (int i = 0; i < SEQ; i++) {
        int32_t row_max = scores[i][0];
        for (int j = 1; j < SEQ; j++)
            if (scores[i][j] > row_max) row_max = scores[i][j];

        int32_t sum = 0;
        for (int j = 0; j < SEQ; j++) {
            int32_t diff = row_max - scores[i][j]; // >= 0
            int32_t shift_amt = diff >> DECAY_SHIFT;
            int32_t w = (shift_amt < 31) ? (WEIGHT_ONE >> shift_amt) : 0;
            weights[i][j] = w;
            sum += w;
        }
        if (sum == 0) sum = 1;
        for (int j = 0; j < SEQ; j++)
            weights[i][j] = (weights[i][j] * WEIGHT_ONE) / sum; // divide
    }
}

// 3. out[i] = sum_j weights[i][j] * V[j]
static void av_matmul(void) {
    for (int i = 0; i < SEQ; i++)
        for (int d = 0; d < DM; d++) {
            int32_t acc = 0;
            for (int j = 0; j < SEQ; j++)
                acc += weights[i][j] * (int32_t)V[j][d];
            out[i][d] = acc / WEIGHT_ONE;
        }
}
```

2 Commands

Set up once, then build and run each application in turn. Each run prints one profiling line per PROFILE_START/PROFILE_END pair — one line for matmul and crc32, three lines for attention.

```
make vendor # pull X-HEEP into ./x-heep
make mcu-gen # generate the MCU from config.py
make verilator-build # build the Verilator model

make app PROJECT=matmul # compile matmul
make verilator-run PROJECT=matmul # run it (prints cycle/instr count)

make app PROJECT=crc32 # compile crc32
make verilator-run PROJECT=crc32 # run it (prints cycle/instr count)

make app PROJECT=tinyconv # compile tinyconv
make verilator-run PROJECT=tinyconv # run it (prints cycle/instr count)
```

```
make app PROJECT=attention # compile attention
make verilator-run PROJECT=attention # run it (prints 3 cycle/instr lines)

make profile # flamegraph from the last run's .fst
make verilator-waves # inspect waveforms (gtkwave)
```

3 Reading the output: an example

Profiling gives you three different views of the same run. Here is what each one actually looks like, and how to read it.

3.1 Cycle-count printf output

This is what `make verilator-run` prints, straight from the `PROFILE_START/PROFILE_END` macros. Here it is for `matmul` and `crc32` on the reference build:

```
$ make app PROJECT=matmul && make verilator-run PROJECT=matmul
[profile] matmul: 68199 cycles, 30580 instr
matmul 16x16 checksum=0x...

$ make app PROJECT=crc32 && make verilator-run PROJECT=crc32
[profile] crc32: 20227 cycles, 15621 instr
crc32 len=256 result=0x...
```

How to read it: `cycles` is how long the bracketed region took on the simulated core; `instr` is how many instructions it dynamically executed to get there. Two things follow directly from these two numbers:

- **Absolute cost.** `matmul` costs about $3.4\times$ more cycles than `crc32` for its kernel — on its own this only tells you which region is more expensive, not why.
- **Cycles per instruction (CPI).** Divide the two: `matmul` is $68199/30580 \approx 2.2$ cycles/instr, `crc32` is $20227/15621 \approx 1.3$ cycles/instr. A CPI close to 1 means the core is mostly retiring one instruction per cycle (compute-bound, few stalls); a higher CPI hints at stalls — waiting on memory, mispredicted branches, or multi-cycle operations (a division, for instance). This is often the first clue about *why* a kernel is slow, before you even open a flamegraph.

For `attention`, the same run instead prints *three* lines, one per sub-kernel, in this format (illustrative values — run it yourself to get real ones):

```
[profile] qk_matmul: <cycles> cycles, <instr> instr
[profile] softmax: <cycles> cycles, <instr> instr
[profile] av_matmul: <cycles> cycles, <instr> instr
attention seq=8 dm=8 checksum=0x...
```

This is the point of splitting the profiling: with one number, there is nothing to decide. With three, you do — and the answer is not obvious from reading the code alone. The two `matmul`s (`qk_matmul`, `av_matmul`) look like the “hardware-shaped” part; but `softmax` does an integer *division* per element, and division is expensive on a small RISC-V core. Whether the `matmul`s or the division-heavy normalization actually dominate is exactly the kind of question you cannot answer by inspection — it is why this lab makes you measure before you design anything. Compare the CPI of the three sub-kernels the same way as above: a `softmax` CPI noticeably higher than the two `matmul`s’ is the profiler telling you, in one number, that the division loop is stalling the core far more per instruction than the MACs are.

3.2 RV_PROFILE flamegraph output

`make profile` turns the waveform from your last run into a flamegraph (`x-heep/util/profile/flamegraph.svg`), an *icicle plot* of which functions were on the call stack, and for how long. Figure 1 sketches what one looks like for `matmul` (illustrative proportions, not measured pixel-for-pixel — open the real SVG to get the real one).

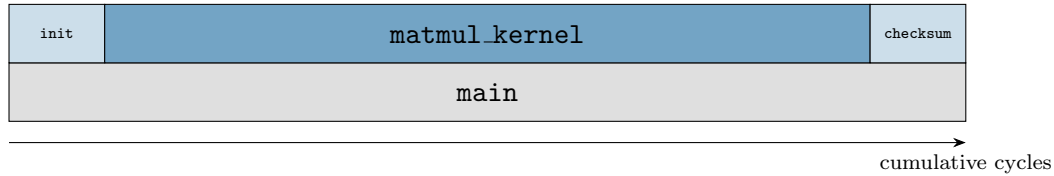


Figure 1: How to read a flamegraph: each row is one level of the call stack (`main` at the bottom, callees stacked above it); the *width* of a box is proportional to the cycles spent in that function (including whatever it calls); *siblings* are ordered left-to-right, not by time of call. The widest box — here `matmul_kernel` — is where to look first.

How to read it: width, not position, is the signal. A wide box means many cycles were attributed to that function; a narrow one means few. Stacking (a box directly above another) means the top function was called from the bottom one. If the box for the function you already suspected from the cycle count (Section 3.1) is indeed the widest one at its level, the two methods agree and you have a confirmed hotspot; if the widest box is a function you did *not* expect, that is the flamegraph earning its keep.

3.3 Waveform inspection

`make verilator-waves` opens the same run’s signals in GTKWave. This is a **debugging aid only** here, not a numeric measurement: use it to sanity-check that the bus/memory activity you see during the region actually lines up with the loop you profiled (e.g., a burst of memory reads exactly while `matmul_kernel` is running), not to extract a new number.

What to do with the numbers

For whichever application you pick, you should be able to point at the profiling output and say, concretely: this function (or this one sub-kernel out of several) is where the cycles go, and here is why its shape makes it a good — or bad — candidate for a hardware accelerator.

References

- [1] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. *Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference*. CVPR, 2018.
- [2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. *Gradient-Based Learning Applied to Document Recognition*. Proceedings of the IEEE, 86(11), 1998.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. *Attention Is All You Need*. NeurIPS, 2017.
- [4] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan. *Softmax: Hardware/Software Co-Design of an Efficient Softmax for Transformers*. DAC, 2021.
- [5] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer. *I-BERT: Integer-only BERT Quantization*. ICML, 2021.



- [6] W. W. Peterson and D. T. Brown. *Cyclic Codes for Error Detection*. Proceedings of the IRE, 49(1), 1961.